КУРСОВАЯ РАБОТА

По дисциплине: Программирование

Тема: «Хранение и обработка числовых данных на основе файлов и массивов»

Автор: Кутепов П.О.

СПбГЭТУ «ЛЭТИ», кафедра САПР

2024

АННОТАЦИЯ

Работа посвящена хранению и обработке числовых данных на основе файлов и массивов. На примере геометрической задачи — поиска пар квадратов с наибольшей площадью пересечения — реализована программа на C++, которая:

- читает точки из текстового файла;
- строит все квадраты с вершинами в этих точках;
- находит пары с максимальной площадью пересечения;
- выводит результат в отдельный файл и ведёт протокол обработки данных.

По требованию задания программа не использует стандартные контейнеры и библиотечные типы: работа с памятью ведётся через сырые указатели и динамические массивы. Исходный код, примеры входных данных и полная пояснительная записка доступны в репозитории.

Структура записки: исходная формулировка задания, математическая постановка, описание алгоритма, листинг программы, контрольный пример, выводы.

Содержание

Исходная формулировка задания 3

Ограничения на исходные данные 3

Способ решения 3

Контрольный пример 6

Организация интерфейса пользователя 6

Работа с файлами 8

Описание функций 8

Описание алгоритма 11

Текст программы 16

Вывод программы 20

Выводы 21

Исходная формулировка задания

Дано N точек на плоскости. Найти все пары квадратов с вершинами в этих точках, имеющих наибольшую площадь пересечения. Вывод вершин фигур осуществить против часовой стрелки. Рассматривать стороны фигур расположенными параллельно. Программа не должна использовать стандартные контейнеры и библиотечные типы: работа с памятью ведётся через сырые указатели и динамические массивы.

Ограничения на исходные данные

Будем обрабатывать не больше 100 точек ($N \leq 100$).

Способ решения

Построим квадраты на плоскости (рис. 1):

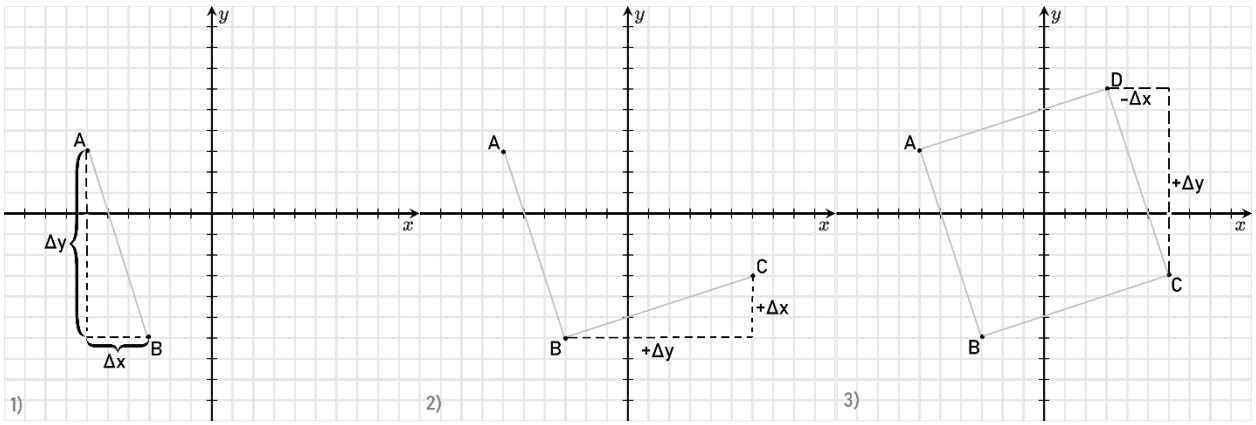


Рисунок 1 – Поэтапно строим квадрат

1. Берём две точки $A = (x_a; y_a)$ и $B = (x_b; y_b)$, причём B удовлетворяет условиям $x_b \geq x_a$ и $y_b < y_a$. Найдём разность координат $\Delta x = x_b - x_a, \Delta y = y_a - y_b$ этих точек.

2. Выбираем следующую точку $C = (x_c; y_c)$, такую, чтобы $x_c = x_b + \Delta y, y_c = y_b + \Delta x$.

3. Последняя точка $D = (x_d, y_d)$ должна удовлетворять условиям $x_d = x_c - \Delta x, y_d = y_c + \Delta y$.

Повторяем пункты 1-3 до тех пор, пока не переберём все N точек. Теперь необходимо «повернуть» квадраты, чтобы их стороны были параллельны (перпендикулярны) осям. Для этого находим угол между стороной квадрата и осью Ox: $\alpha = \tan^{-1}\frac{\Delta x}{\Delta y}$ (рис. 2). Далее поворачиваем вектор по формуле $x' = x\cos(-\alpha) - y\sin(-\alpha)$, $y' = x\sin(-\alpha) + y\cos(-\alpha)$.

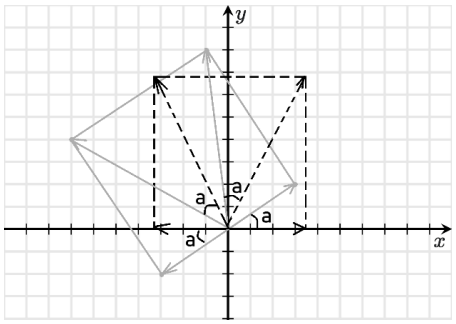


Рисунок 2 – «Поворот» квадрата

Далее находим площадь пересечения. Обозначим за l, w и S длину пересечения (по оси Ox), ширину пересечения (по оси Oy) и площадь пересечения квадратов соответственно. Далее рассмотрим 3 случая (рис. 3):

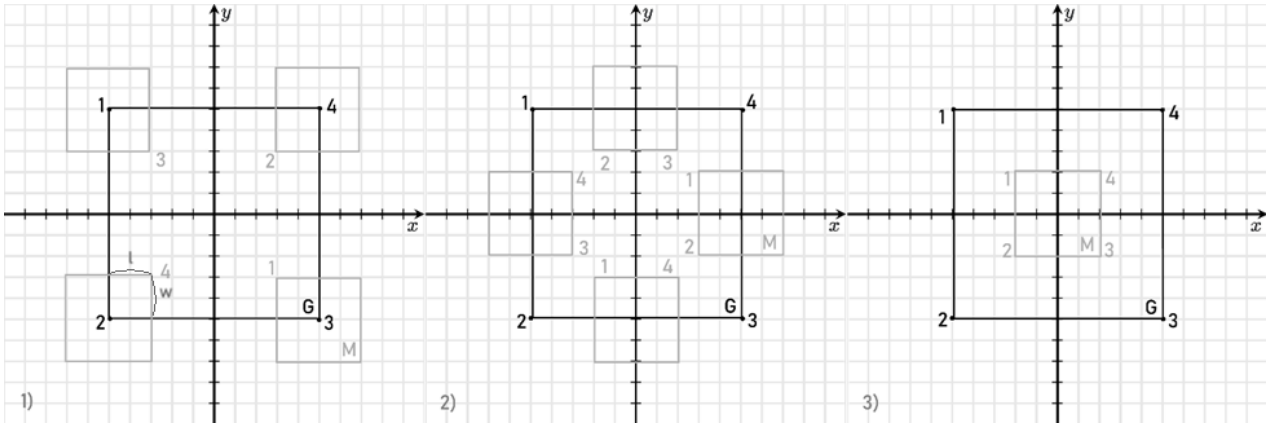


Рисунок 3 – 3 случая пересечений квадратов

1) Квадрат M «заходит» на G одной вершиной. В этом случае $S = |(x_{mi} - x_{gj}) \times (y_{mi} - y_{gj})|$, $j = ((1 + i) mod\ 4) + 1$. mi, gj – вершины M и G соответственно, $i, j \in \mathbb{N}$ – индексы вершин.

2) Квадрат M «заходит» на G двумя вершинами. Если вершины 1, 4 или 2, 3, то $S = |(x_{m4} - x_{m1}) \times (y_{mi} - y_{gj})|$, $j = |i - 2| + 1$, $(i \in \{1\} \cup \{2\})$. Если вершины 4, 3 или 1, 2, то $S = |(x_{mi} - x_{gj}) \times (y_{m4} - y_{m3})|$, $j = |i - 3| + 1$, $(i \in \{1\} \cup \{3\})$.

3) Квадрат M «заходит» на G четырьмя вершинами. $S = (x_{m4} - x_{m1}) \times (y_{m1} - y_{m2})$.

Среди всех значений S ответом будет наибольшее значение S_{max}.

Контрольный пример

(рис 3)**:** $S_{max} = 2{,}5$ у квадратов с вершинами (-1;2)(0;- 1)(3;0)(2;3) и (-7;5)(-5;-1)(1;1)(-1;7).

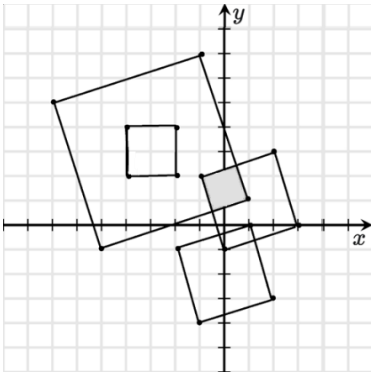


Рисунок 4 – Контрольный пример

Организация интерфейса пользователя

Макет ввода/вывода

Fresult1.	(d__d.d__d;d__d.d__d), (d__d.d__d;d__d.d__d), (d__d.d__d;d__d.d__d), (d__d.d__d;d__d.d__d)
Fresult2.	ЗАДАНИЕ: Дано N точек на плоскости. Найти все пары квадратов с вершинами в этих точках, имеющих наибольшую площадь пересечения. Вывод вершин осуществить против часовой стрелки. Стороны квадратов расположены параллельно. ОБРАБОТКА ДАННЫХ:
Fresult3.	Кол-во точек = 0.
Fresult4.	На обработку взято d__d точек. Построено d__d квадратов.
Fresult5.	РЕЗУЛЬТАТ:
Fresult6.	Наибольшая площадь пересечения, равная = d__d, у
Fresult7.	квадрата с вершинами в точках
Fresult8.	и квадрата с вершинами
Fresult9.	Никакие квадраты не пересекаются.
Fresult10.	Файл пуст.
Fresult11.	Кол-во точек не может быть отрицательным.
Fresult12.	Кол-во точек = 0.
Fresult13.	Слишком большое кол-во точек.
Fresult14.	Не удалось открыть протокол.
Fresult15.	Не удалось открыть файл с входными данными.
Fprotocol1.	Пустая строка.
Fprotocol2.	Не удалось взять точку (d__d.d__d;?). Недостаточно данных.

Fprotocol3.	Получена d__d.d__d -я точка с координатами (d__d.d__d;d__d.d__d)
Fprotocol4.	Точка (d__d.d__d;d__d.d__d) уже записана.
Fprotocol5.	Построен d__d-й квадрат с вершинами
Fprotocol6.	Не удалось построить d__d-й квадрат с вершинами
Fprotocol7.	Площадь пересечения d__d-го и d__d-го квадратов равна =
Fprotocol8.	d__d-й и d__d-й квадраты не пересекаются.
Cout1.	Не удалось открыть файл для вывода результата.
Fpoints1.	s_s
Fpoints2.	d__d.d__d

Реализация вывода в консоль

Библиотека	Вывод
iostream	std::cout <<

Внутренний формат хранения данных

Имя	Тип	Назначение
max	float	Максимальная площадь пересечения
i, j, p	short	Вспомогательные
sqr1, sqr2	short	Номера пересекающихся квадратов
tg1, tg2	float	Тангенс угла между стороной квадрата и осью координат
pntnum	short[]	Сохраняет номера точек пересечения фигур
intrsctn	short	Кол-во точек, по которым пересекаются фигуры
sym	char	Символ из файла
crd	short	Номер координаты (0 = x, 1 = y)
flag_same	bool	Определяет, есть ли схожие элементы в массиве
same	short	Для поиска одинаковых элементов
angle	float	Угол между стороной квадрата и осью Ox
top[4][2]	float	Массив для хранения вершин квадрата
sqr, sqrn	short	Номер квадрата
pnt	short	Номер вершины квадрата
top1, top2, top3, top4	short	Предполагаемые вершины квадрата
deltaX, deltaY	float	Разность координат между первой и второй предполагаемыми вершинами квадрата
l, w	float	Длинна и ширина прямоугольника, по которому пересекаются квадраты
area	float	Площадь пересечения
N0	short	Кол-во точек, введённое пользователем
N	short	Реальное кол-во точек
ArrP	float**	Массив для хранения точек
ArrS	float***	Массив для хранения квадратов

count_s	short	Кол-во построенных квадратов
ArrA	float**	Массив для хранения площадей пересечения фигур
count_a	short	Кол-во пересекающихся фигур

Работа с файлами

Для работы с файлами используется библиотека «fstream».

Метод	Описание
fstream f	Создание файловой переменной
f.open(“file.txt”, ios::out)	Открытие файла на запись
f.open(“file.txt”, ios::in)	Открытие файла на чтение
filein.seekg(-1, ios::cur);	Перемещение курсора назад относительно текущего положения
filein >> noskipws	Чтение пробелов, символов перехода на новую строку
f.is_open()	Определяет, открыт ли файл
f.eof()	Проверка на конец файла
f <<	Запись данных в файл
f >>	Чтение данных из файла

Описание функций

Синтаксис

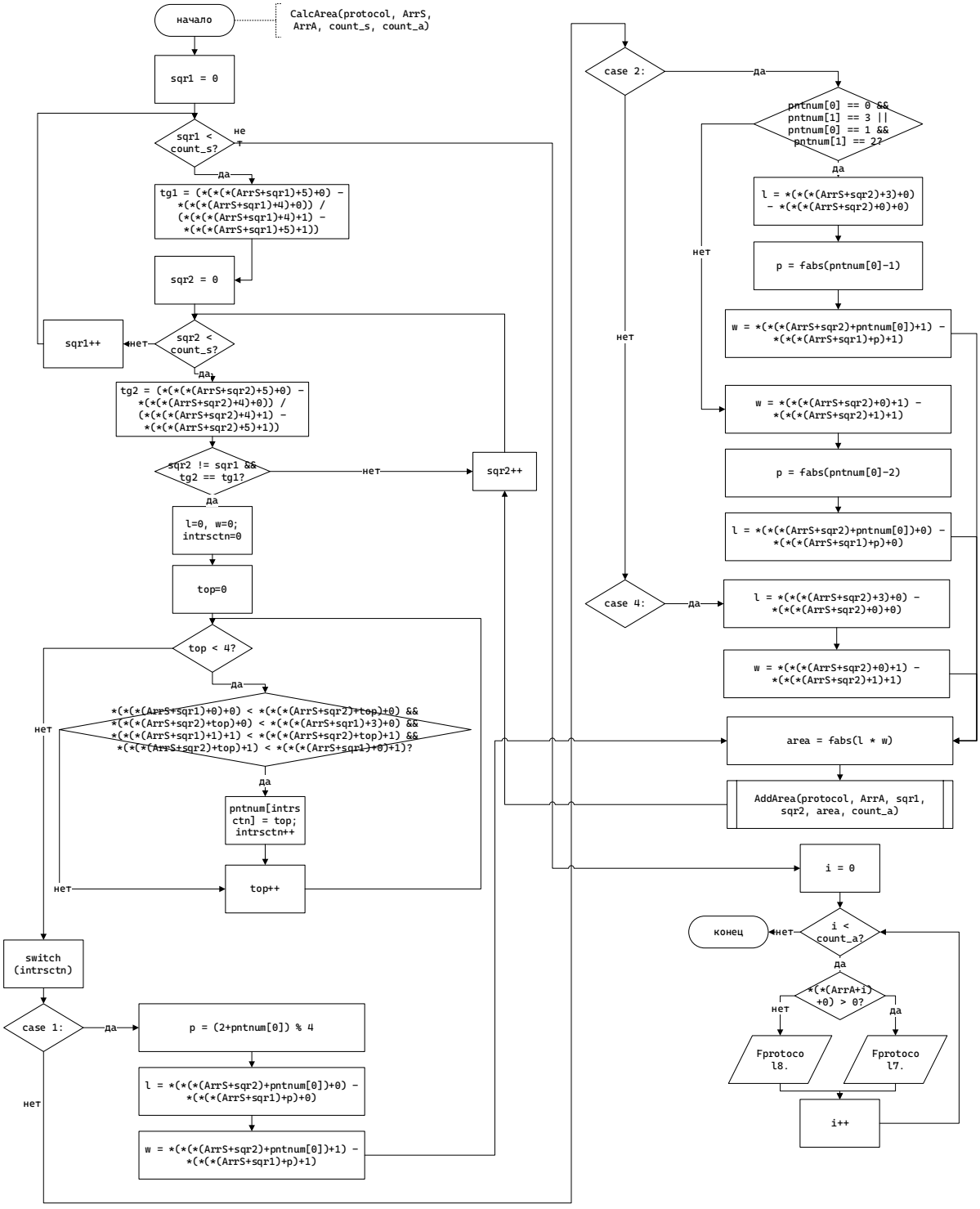
Имя	Тип возвращаемого значения	Параметры		
		Тип	Имя	Изменение
PrintCrd	void	ofstream	file	&
		float	Arr	**
		short	pnt1, pnt2, pnt3, pnt4	-
PrintRes	void	float	ArrA	**
		float	ArrS	***
		short	count_s, count_a, N	-
		ofstream	result	&
CalcSize	bool	ifstream	points	&
		ofstream	result	&
		short	N0	&
GetPoints	void	ifstream	points	&
		ofstream	protocol	&
		float	ArrP	**
		short	N0	-

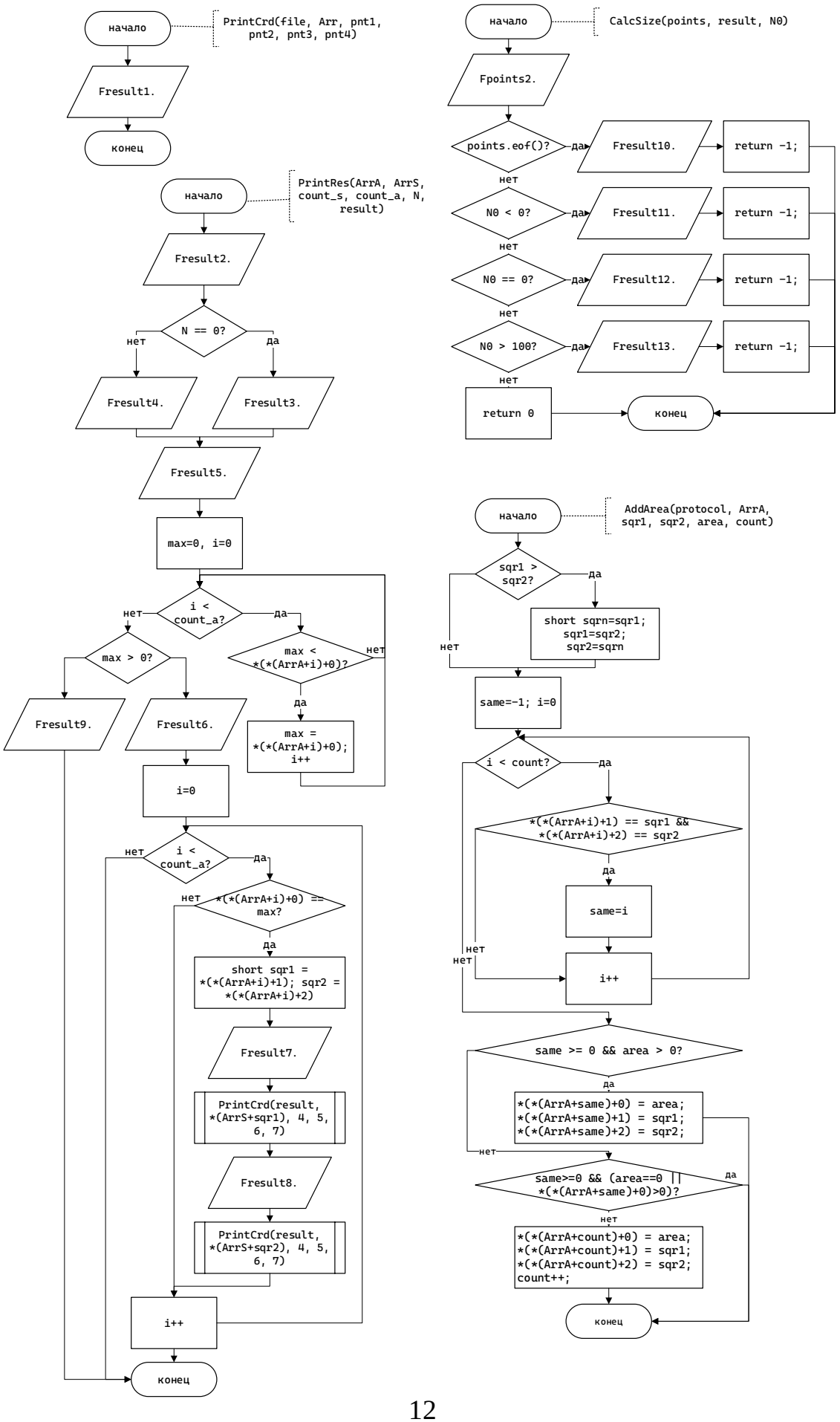
		short	N	&
Transform	void	float[]	pnt	&
		float	ArrP	**
		short	nump	-
		float	angle	-
AddSquare	bool	float	ArrS	***
		float	ArrP	**
		short	count	&
		short	pnt1, pnt2, pnt3, pnt4	-
GetSquare	void	ofstream	protocol	&
		float	ArrP	**
		float	ArrS	***
		short	N	-
		short	count_s	&
AddArea	void	float	ArrA	**
		short	sqr1, sqr2	-
		float	area	-
		short	count	&
CalcArea	void	ofstream	protocol	&
		float	ArrS	***
		float	ArrA	**
		short	count_s	-
		short	count_a	&
main	void	-	-	-

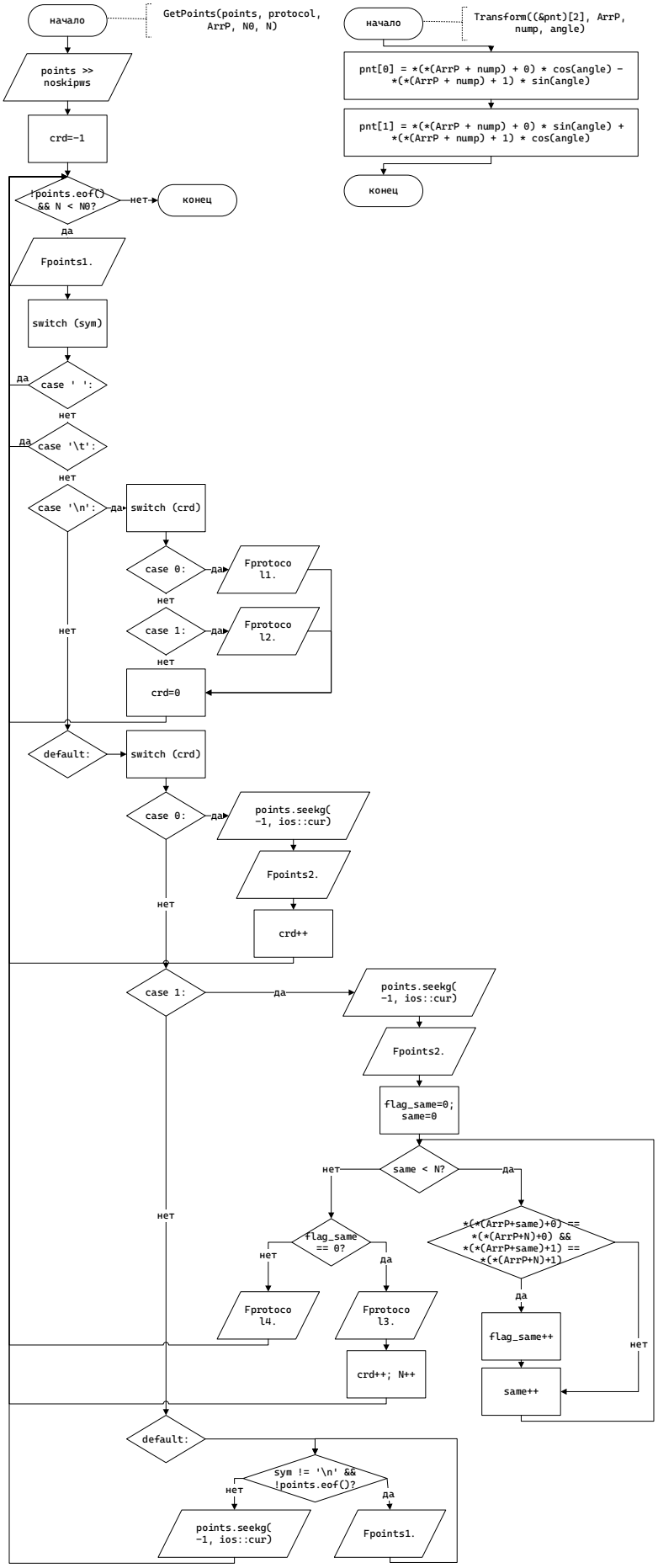
Имя	Возвращ. знач.	Внешние изменения	Параметры				Назначение
			Входной	Выходной	Модифицируемый	Транзитный	
PrintCrd	-	-	Arr, pnt1, pnt2, pnt3, pnt4	-	file	-	Выводит в файл координаты точки
PrintRes	-	-	ArrA, ArrS, count_s, count_a, N	max, i, sqr1, sqr2	result	-	Выводит результат в файл
CalcSize	1/0	-	points	-	result, N0	-	Определяет число точек в файле
GetPoints	-	-	points, N0	sym, crd, flag_same, same	protocol, ArrP, N	-	Сохраняет точки в массив
Transform	-	-	ArrP, nump, angle	-	pnt[]	-	Поворачивает вектор на заданный угол
AddSquare	0/1	-	ArrS, ArrP, pnt1, pnt2, pnt3, pnt4	angle, top[][],same, sqr, pnt, crd	count	-	Сохраняет вершины квадрата в массив
GetSquare	-	-	ArrP, N	top1, top1, top1, top1, deltaX, deltaY	protocol	ArrS, count_s	Строит квадраты по точкам
AddArea	-	-	sqr1, sqr2, area	sqrn, same, i	ArrA, count	-	Сохраняет площади пересечения фигур в массив
CalcArea	-	-	ArrS, count_s	sqr1, tg1, sqr2, tg2, pntnum[], intrsctn, l, w, p, area, i	protocol	ArrS, count_a	Вычисляет площадь пересечения фигур
main	-	Открытие файлов "result.txt", "protocol.txt" и "points.txt". Зарезервирована память для 3-х массивов	-	i, j, N0, N, ArrP, ArrS, ArrA, result, protocol, points	-	-	Основная функция программы

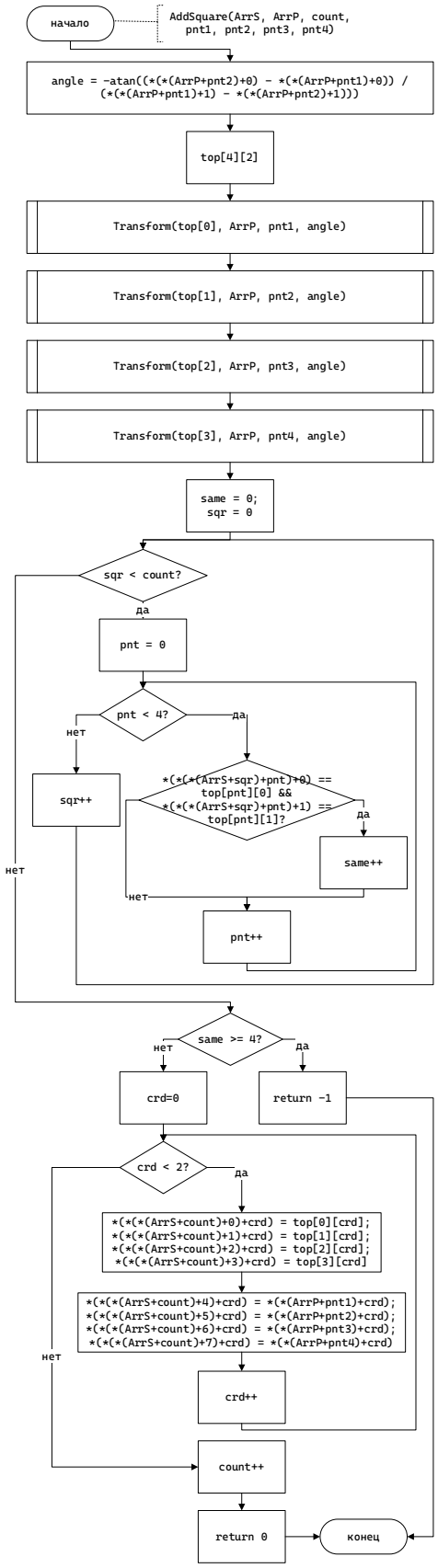
Описание алгоритма

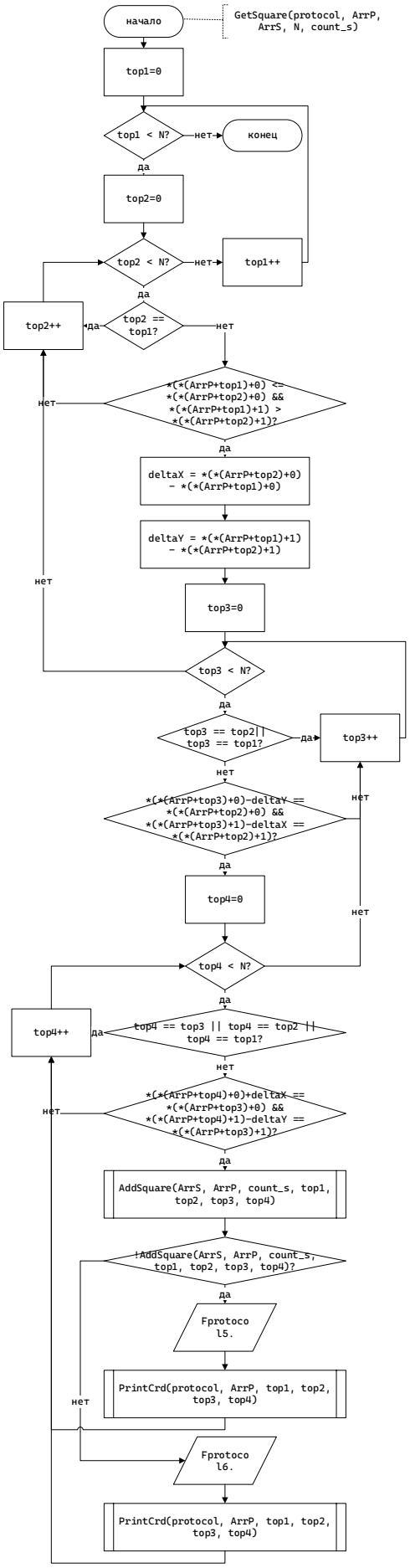
Программа определяет кол-во точек в файле «points.txt», затем считывает точки из входного файла. Далее строятся квадраты, находится их площадь пересечения. Результат работы программы выводится в файл «result.txt». Все действия программы фиксируются в протоколе «protocol.txt». Блок-схема работы программы представлена ниже:

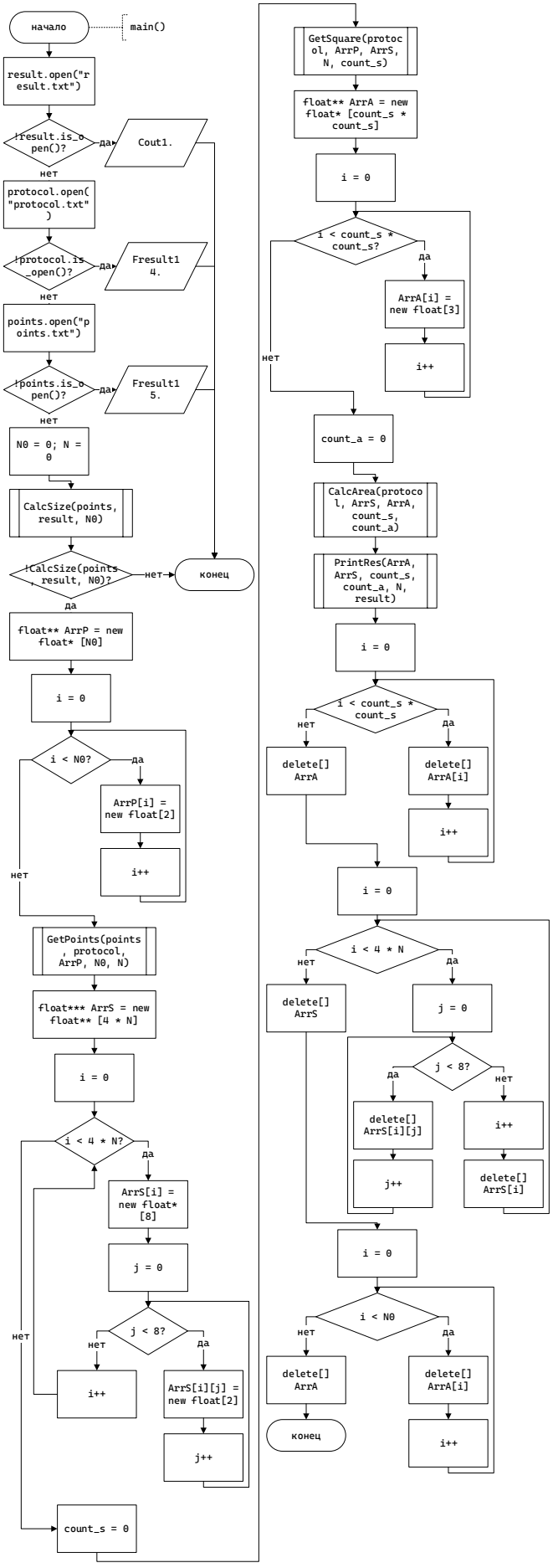



начало
PrintCrd(file, Arr, pnt1, pnt2, pnt3, pnt4)
Fresult1.
конец
начало
PrintRes(ArrA, ArrS, count_s, count_a, N, result)
Fresult2.
N == 0?
нет
да
Fresult4.
Fresult3.
Fresult5.
max=0, i=0
i < count_a?
нет
да
max > 0?
max < *(*(ArrA+i)+0)?
нет
да
Fresult9.
Fresult6.
max = *(*(ArrA+i)+0); i++
i=0
i < count_a?
нет
да
((ArrA+i)+0) == max?
нет
да
short sqr1 = *(*(ArrA+i)+1); sqr2 = *(*(ArrA+i)+2)
Fresult7.
PrintCrd(result, *(ArrS+sqr1), 4, 5, 6, 7)
Fresult8.
PrintCrd(result, *(ArrS+sqr2), 4, 5, 6, 7)
i++
конец
начало
CalcSize(points, result, N0)
Fpoints2.
points.eof()?
да
Fresult10.
return -1;
нет
N0 < 0?
да
Fresult11.
return -1;
нет
N0 == 0?
да
Fresult12.
return -1;
нет
N0 > 100?
да
Fresult13.
return -1;
нет
return 0
конец
начало
AddArea(protocol, ArrA, sqr1, sqr2, area, count)
sqr1 > sqr2?
да
short sqrn=sqr1; sqr1=sqr2; sqr2=sqrn
нет
same=-1; i=0
i < count?
да
((ArrA+i)+1) == sqr1 && *(*(ArrA+i)+2) == sqr2
да
same=i
нет
нет
i++
same >= 0 && area > 0?
да
((ArrA+same)+0) = area;
((ArrA+same)+1) = sqr1;
((ArrA+same)+2) = sqr2;
нет
same>=0 && (area==0 || *(*(ArrA+same)+0)>0)?
да
нет
((ArrA+count)+0) = area;
((ArrA+count)+1) = sqr1;
((ArrA+count)+2) = sqr2;
count++;
конец


начало
GetPoints(points, protocol, ArrP, N0, N)
points >> noskipws
crd=-1
!points.eof() && N < N0?
нет
конец
да
Fpoints1.
switch (sym)
case ' ':
да
нет
case '\t':
да
нет
case '\n':
да
switch (crd)
case 0:
да
Fprotocol1.
нет
case 1:
да
Fprotocol2.
нет
crd=0
нет
default:
switch (crd)
case 0:
да
points.seekg(-1, ios::cur)
Fpoints2.
crd++
нет
case 1:
да
points.seekg(-1, ios::cur)
Fpoints2.
flag_same=0; same=0
same < N?
нет
да
flag_same == 0?
нет
да
Fprotocol4.
Fprotocol3.
crd++; N++
((ArrP+same)+0) == *(*(ArrP+N)+0) && *(*(ArrP+same)+1) == *(*(ArrP+N)+1)
да
flag_same++
same++
нет
нет
default:
sym != '\n' && !points.eof()?
нет
да
points.seekg(-1, ios::cur)
Fpoints1.
начало
Transform((&pnt)[2], ArrP, nump, angle)
pnt[0] = *(*(ArrP + nump) + 0) * cos(angle) - *(*(ArrP + nump) + 1) * sin(angle)
pnt[1] = *(*(ArrP + nump) + 0) * sin(angle) + *(*(ArrP + nump) + 1) * cos(angle)
конец


начало
AddSquare(ArrS, ArrP, count, pnt1, pnt2, pnt3, pnt4)
angle = -atan((*(*(ArrP+pnt2)+0) - *(*(ArrP+pnt1)+0)) / (*(*(ArrP+pnt1)+1) - *(*(ArrP+pnt2)+1)))
top[4][2]
Transform(top[0], ArrP, pnt1, angle)
Transform(top[1], ArrP, pnt2, angle)
Transform(top[2], ArrP, pnt3, angle)
Transform(top[3], ArrP, pnt4, angle)
same = 0; sqr = 0
sqr < count?
да
нет
pnt = 0
pnt < 4?
да
нет
sqr++
((*(ArrS+sqr)+pnt)+0) == top[pnt][0] && *(*(*(ArrS+sqr)+pnt)+1) == top[pnt][1]?
да
нет
same++
pnt++
same >= 4?
нет
да
crd=0
return -1
crd < 2?
да
нет
((*(ArrS+count)+0)+crd) = top[0][crd];
((*(ArrS+count)+1)+crd) = top[1][crd];
((*(ArrS+count)+2)+crd) = top[2][crd];
((*(ArrS+count)+3)+crd) = top[3][crd]
((*(ArrS+count)+4)+crd) = *(*(ArrP+pnt1)+crd);
((*(ArrS+count)+5)+crd) = *(*(ArrP+pnt2)+crd);
((*(ArrS+count)+6)+crd) = *(*(ArrP+pnt3)+crd);
((*(ArrS+count)+7)+crd) = *(*(ArrP+pnt4)+crd)
crd++
count++
return 0
конец


начало
GetSquare(protocol, ArrP, ArrS, N, count_s)
top1=0
top1 < N?
нет
конец
да
top2=0
top2 < N?
нет
top1++
да
top2++
да
top2 == top1?
нет
нет
((ArrP+top1)+0) <= *(*(ArrP+top2)+0) && *(*(ArrP+top1)+1) > *(*(ArrP+top2)+1)?
да
deltaX = *(*(ArrP+top2)+0) - *(*(ArrP+top1)+0)
deltaY = *(*(ArrP+top1)+1) - *(*(ArrP+top2)+1)
нет
top3=0
top3 < N?
да
top3 == top2|| top3 == top1?
да
top3++
нет
нет
((ArrP+top3)+0)-deltaY == *(*(ArrP+top2)+0) && *(*(ArrP+top3)+1)-deltaX == *(*(ArrP+top2)+1)?
да
нет
top4=0
top4 < N?
да
top4++
да
top4 == top3 || top4 == top2 || top4 == top1?
нет
нет
((ArrP+top4)+0)+deltaX == *(*(ArrP+top3)+0) && *(*(ArrP+top4)+1)-deltaY == *(*(ArrP+top3)+1)?
да
AddSquare(ArrS, ArrP, count_s, top1, top2, top3, top4)
!AddSquare(ArrS, ArrP, count_s, top1, top2, top3, top4)?
да
Fprotocol5.
PrintCrd(protocol, ArrP, top1, top2, top3, top4)
нет
Fprotocol6.
PrintCrd(protocol, ArrP, top1, top2, top3, top4)


начало
main()
result.open("result.txt")
!result.is_open()?
да
Cout1.
нет
protocol.open("protocol.txt")
!protocol.is_open()?
да
Fresult14.
нет
points.open("points.txt")
!points.is_open()?
да
Fresult15.
нет
N0 = 0; N = 0
CalcSize(points, result, N0)
!CalcSize(points, result, N0)?
нет
конец
да
float** ArrP = new float* [N0]
i = 0
i < N0?
да
ArrP[i] = new float[2]
i++
нет
GetPoints(points, protocol, ArrP, N0, N)
float*** ArrS = new float** [4 * N]
i = 0
i < 4 * N?
да
ArrS[i] = new float* [8]
j = 0
j < 8?
нет
да
i++
ArrS[i][j] = new float[2]
j++
нет
count_s = 0
GetSquare(protocol, ArrP, ArrS, N, count_s)
float** ArrA = new float* [count_s * count_s]
i = 0
i < count_s * count_s?
да
ArrA[i] = new float[3]
i++
нет
count_a = 0
CalcArea(protocol, ArrS, ArrA, count_s, count_a)
PrintRes(ArrA, ArrS, count_s, count_a, N, result)
i = 0
i < count_s * count_s
нет
да
delete[] ArrA
delete[] ArrA[i]
i++
i = 0
i < 4 * N
нет
да
delete[] ArrS
j = 0
j < 8?
да
нет
delete[] ArrS[i][j]
j++
i++
delete[] ArrS[i]
i = 0
i < N0
нет
да
delete[] ArrA
delete[] ArrA[i]
конец
i++

Текст программы

```
#include <iostream>
#include <fstream>
#include <cmath>
using namespace std;

void PrintCrd(ofstream& file, float** Arr, short pnt1, short pnt2, short pnt3, short pnt4) {

    file << "(" << *(*(Arr + pnt1) + 0) << ";" << *(*(Arr + pnt1) + 1) << "), ("
        << *(*(Arr + pnt2) + 0) << ";" << *(*(Arr + pnt2) + 1) << "), ("
        << *(*(Arr + pnt3) + 0) << ";" << *(*(Arr + pnt3) + 1) << "), ("
        << *(*(Arr + pnt4) + 0) << ";" << *(*(Arr + pnt4) + 1) << ")";
}

void PrintRes(float** ArrA, float*** ArrS, short count_s, short count_a, short N, ofstream& result) {

    result << "ЗАДАНИЕ:\n"
        << "Дано N точек на плоскости. Найти все пары квадратов с вершинами в этих точках,\n"
        << "имеющих наибольшую площадь пересечения. Вывод вершин осуществить против часовой\n"
        << "стрелки. Стороны квадратов расположены параллельно.\n\n"
        << "ОБРАБОТКА ДАННЫХ:\n";
    if (N == 0) { result << "Кол-во точек = 0."; return; }
    else result << "На обработку взято " << N << " точек." << "\nПостроено " << count_s << "
квадратов.\n\n";
    result << "РЕЗУЛЬТАТ:\n";

    float max = 0;
    for (short i = 0; i < count_a; i++) {
        if (max < *(*(ArrA + i) + 0)) max = *(*(ArrA + i) + 0);
    }
    if (max > 0) {
        result << "Наибольшая площадь пересечения, равная = " << max << ", у\n";

        for (short i = 0; i < count_a; i++) {
            if (*(*(ArrA + i) + 0) == max) {
                short sqr1 = *(*(ArrA + i) + 1);
                short sqr2 = *(*(ArrA + i) + 2);
                result << "квадрата с вершинами в точках ";
                PrintCrd(result, *(ArrS + sqr1), 4, 5, 6, 7);
                result << "\nи квадрата с вершинами ";
                PrintCrd(result, *(ArrS + sqr2), 4, 5, 6, 7);
                result << "\n\n";
            }
        }
    }
    else result << "Никакие квадраты не пересекаются.\n";
}

bool CalcSize(ifstream& points, ofstream& result, short& N0) {

    points >> N0;
    if (points.eof()) { result << "Файл пуст."; return -1; }
    else if (N0 < 0) { result << "Кол-во точек не может быть отрицательным."; return -1; }
    else if (N0 == 0) { result << "Кол-во точек = 0."; return -1; }
    else if (N0 > 100) { result << "Слишком большое кол-во точек."; return -1; }
    return 0;
}

void GetPoints(ifstream& points, ofstream& protocol, float** ArrP, short N0, short& N) {

    points >> noskipws;
    char sym; short crd = -1;
    while (!points.eof() && N < N0) {
        points >> sym;
        switch (sym) {
        case ' ': { continue; break; }
        case '\t': { continue; break; }
        case '\n': {
            switch (crd) {
            case 0: {
                protocol << "Пустая строка.\n";
                break;
            }
```

```
            case 1: {
                protocol << "Не удалось взять точку (" << *(*(ArrP + N) + 0) << ";?). Недостаточно
данных.\n"; break;
            }
            }
            crd = 0;
            continue; break;
        }
        default: {
            switch (crd) {
            case 0: {
                points.seekg(-1, ios::cur);
                points >> *(*(ArrP + N) + crd);
                crd++;
                continue; break;
            }
            case 1: {
                points.seekg(-1, ios::cur);
                points >> *(*(ArrP + N) + crd);
                bool flag_same = 0;
                for (short same = 0; same < N; same++) {
                    if (*(*(ArrP + same) + 0) == *(*(ArrP + N) + 0) && *(*(ArrP + same) + 1) == *(*(ArrP +
N) + 1)) flag_same = 1;
                }
                if (flag_same == 0) {
                    protocol << "Получена " << N + 1 << "-я точка с координатами (" << *(*(ArrP + N) + 0)
<< ";" << *(*(ArrP + N) + 1) << ")\n";
                    crd++;
                    N++;
                }
                else protocol << "Точка (" << *(*(ArrP + N) + 0) << ";" << *(*(ArrP + N) + 1) << ") уже
записана.\n"; crd = 2;
                continue; break;

            }
            default: {
                while (sym != '\n' && !points.eof()) points >> sym;
                points.seekg(-1, ios::cur);
                continue; break;
            }
            }
        }
        }
    }
}

void Transform(float (&pnt)[2], float** ArrP, short nump, float angle) {

    pnt[0] = *(*(ArrP + nump) + 0) * cos(angle) - *(*(ArrP + nump) + 1) * sin(angle);
    pnt[1] = *(*(ArrP + nump) + 0) * sin(angle) + *(*(ArrP + nump) + 1) * cos(angle);
}

bool AddSquare(float*** ArrS, float** ArrP, short& count, short pnt1, short pnt2, short pnt3, short pnt4)
{

    float angle = -atan((*(*(ArrP + pnt2) + 0) - *(*(ArrP + pnt1) + 0)) / (*(*(ArrP + pnt1) + 1) -
*(*(ArrP + pnt2) + 1)));
    float top[4][2];
    Transform(top[0], ArrP, pnt1, angle);
    Transform(top[1], ArrP, pnt2, angle);
    Transform(top[2], ArrP, pnt3, angle);
    Transform(top[3], ArrP, pnt4, angle);

    short same = 0;
    for (short sqr = 0; sqr < count; sqr++) {
        for (short pnt = 0; pnt < 4; pnt++) {
            if (*(*(*(ArrS + sqr) + pnt) + 0) == top[pnt][0] && *(*(*(ArrS + sqr) + pnt) + 1) ==
top[pnt][1]) same++;
        }
    }

    if (same >= 4) return -1;
    else {
        for (short crd = 0; crd < 2; crd++) {
            *(*(*(ArrS + count) + 0) + crd) = top[0][crd];
```

```
            *(*(*(ArrS + count) + 1) + crd) = top[1][crd];
            *(*(*(ArrS + count) + 2) + crd) = top[2][crd];
            *(*(*(ArrS + count) + 3) + crd) = top[3][crd];
            *(*(*(ArrS + count) + 4) + crd) = *(*(ArrP + pnt1) + crd);
            *(*(*(ArrS + count) + 5) + crd) = *(*(ArrP + pnt2) + crd);
            *(*(*(ArrS + count) + 6) + crd) = *(*(ArrP + pnt3) + crd);
            *(*(*(ArrS + count) + 7) + crd) = *(*(ArrP + pnt4) + crd);
        }
        count++;
        return 0;
    }
}
void GetSquare(ofstream& protocol, float** ArrP, float*** ArrS, short N, short& count_s) {

    for (short top1 = 0; top1 < N; top1++) {
        for (short top2 = 0; top2 < N; top2++) {
            if (top2 == top1) continue;
            if (*(*(ArrP + top1) + 0) <= *(*(ArrP + top2) + 0)&& *(*(ArrP + top1) + 1) > *(*(ArrP + top2)
+ 1)) {
                float deltaX = *(*(ArrP + top2) + 0) - *(*(ArrP + top1) + 0);
                float deltaY = *(*(ArrP + top1) + 1) - *(*(ArrP + top2) + 1);
                for (short top3 = 0; top3 < N; top3++) {
                    if (top3 == top2|| top3 == top1) continue;
                    if (*(*(ArrP + top3) + 0) - deltaY == *(*(ArrP + top2) + 0) && *(*(ArrP + top3) + 1) -
deltaX == *(*(ArrP + top2) + 1)) {
                        for (short top4 = 0; top4 < N; top4++) {
                            if (top4 == top3 || top4 == top2 || top4 == top1) continue;
                            if (*(*(ArrP + top4) + 0) + deltaX == *(*(ArrP + top3) + 0) && *(*(ArrP +
top4) + 1) - deltaY == *(*(ArrP + top3) + 1)) {
                                if (!AddSquare(ArrS, ArrP, count_s, top1, top2, top3, top4)) {
                                    protocol << "Построен " << count_s << "-й квадрат с вершинами ";
                                    PrintCrd(protocol, ArrP, top1, top2, top3, top4); protocol << ".\n";
                                }
                            }
                            else {
                                protocol << "Не удалось построить " << count_s + 1 << "-й квадрат с
вершинами ";
                                PrintCrd(protocol, ArrP, top1, top2, top3, top4); protocol << ".\n";
                            }
                        }
                    }
                }
            }
        }
    }
}

void AddArea(ofstream& protocol, float** ArrA, short sqr1, short sqr2, float area, short& count) {

    if (sqr1 > sqr2) {
    short sqrn = sqr1;
    sqr1 = sqr2;
    sqr2 = sqrn;
}
short same = -1;
for (short i = 0; i < count; i++) {
    if (*(*(ArrA + i) + 1) == sqr1 && *(*(ArrA + i) + 2) == sqr2) same = i;
}
if (same >= 0 && area > 0) {
    *(*(ArrA + same) + 0) = area;
    *(*(ArrA + same) + 1) = sqr1;
    *(*(ArrA + same) + 2) = sqr2;
}
else if (same >= 0 && (area == 0 || *(*(ArrA + same) + 0)>0)) return;
else {
    *(*(ArrA + count) + 0) = area;
    *(*(ArrA + count) + 1) = sqr1;
    *(*(ArrA + count) + 2) = sqr2;
    count++;
}
}

void CalcArea(ofstream& protocol, float*** ArrS, float** ArrA, short count_s, short& count_a) {

    for (short sqr1 = 0; sqr1 < count_s; sqr1++) {
```

```
        float tg1 = (*(*(*(ArrS + sqr1) + 5) + 0) - *(*(*(ArrS + sqr1) + 4) + 0)) / (*(*(*(ArrS + sqr1) +
4) + 1) - *(*(*(ArrS + sqr1) + 5) + 1));
        for (short sqr2 = 0; sqr2 < count_s; sqr2++) {
            float tg2 = (*(*(*(ArrS + sqr2) + 5) + 0) - *(*(*(ArrS + sqr2) + 4) + 0)) / (*(*(*(ArrS +
sqr2) + 4) + 1) - *(*(*(ArrS + sqr2) + 5) + 1));
            if (sqr2 != sqr1 && tg2 == tg1) {
                float l = 0, w = 0;
                short pntnum[4]{}; short intrsctn = 0;
                for (short top = 0; top < 4; top++) {
                    if (*(*(*(ArrS + sqr1) + 0) + 0) < *(*(*(ArrS + sqr2) + top) + 0) && *(*(*(ArrS +
sqr2) + top) + 0) < *(*(*(ArrS + sqr1) + 3) + 0)
                        && *(*(*(ArrS + sqr1) + 1) + 1) < *(*(*(ArrS + sqr2) + top) + 1) && *(*(*(ArrS +
sqr2) + top) + 1) < *(*(*(ArrS + sqr1) + 0) + 1)) {
                        pntnum[intrsctn] = top; intrsctn++;
                    }
                }
                switch (intrsctn) {
                case 1: {
                    short p = (2 + pntnum[0]) % 4;
                    l = *(*(*(ArrS + sqr2) + pntnum[0]) + 0) - *(*(*(ArrS + sqr1) + p) + 0);
                    w = *(*(*(ArrS + sqr2) + pntnum[0]) + 1) - *(*(*(ArrS + sqr1) + p) + 1);
                    break;
                }
                case 2: {
                    if (pntnum[0] == 0 && pntnum[1] == 3 || pntnum[0] == 1 && pntnum[1] == 2) {
                        l = *(*(*(ArrS + sqr2) + 3) + 0) - *(*(*(ArrS + sqr2) + 0) + 0);
                        short p = fabs(pntnum[0] - 1);
                        w = *(*(*(ArrS + sqr2) + pntnum[0]) + 1) - *(*(*(ArrS + sqr1) + p) + 1);
                    }
                    else {
                        w = *(*(*(ArrS + sqr2) + 0) + 1) - *(*(*(ArrS + sqr2) + 1) + 1);
                        short p = fabs(pntnum[0] - 2);
                        l = *(*(*(ArrS + sqr2) + pntnum[0]) + 0) - *(*(*(ArrS + sqr1) + p) + 0);
                    }
                    break;

                }
                case 4: {
                    l = *(*(*(ArrS + sqr2) + 3) + 0) - *(*(*(ArrS + sqr2) + 0) + 0);
                    w = *(*(*(ArrS + sqr2) + 0) + 1) - *(*(*(ArrS + sqr2) + 1) + 1);
                    break;

                }
                }
                float area = fabs(l * w);
                AddArea(protocol, ArrA, sqr1, sqr2, area, count_a);
            }
        }
    }
    for (short i = 0; i < count_a; i++) {
        if (*(*(ArrA+i)+0) > 0) protocol << "Площадь пересечения " << *(*(ArrA + i) + 1) + 1 << "-го и "
<< *(*(ArrA + i) + 2) + 1 << "-го квадратов равна = " << *(*(ArrA + i) + 0) << "\n";
        else protocol << *(*(ArrA + i) + 1) + 1 << "-й и " << *(*(ArrA + i) + 2) + 1 << "-й квадраты не
пересекаются.\n";
    }
}

int main() {

    setlocale(LC_ALL, "RU");

    ofstream result; result.open("result.txt");
    if (!result.is_open()) { cout << "Не удалось открыть файл для вывода результата."; return; }
    ofstream protocol; protocol.open("protocol.txt");
    if (!protocol.is_open()) { result << "Не удалось открыть протокол."; return; }
    ifstream points; points.open("points.txt");
    if (!points.is_open()) { result << "Не удалось открыть файл с входными данными."; return; }

    short N0 = 0; short N = 0;
    if (!CalcSize(points, result, N0)) {
        float** ArrP = new float* [N0];
        for (short i = 0; i < N0; i++) ArrP[i] = new float[2];

        GetPoints(points, protocol, ArrP, N0, N);
```

```
        float*** ArrS = new float** [4 * N];
        for (short i = 0; i < 4 * N; i++) {
            ArrS[i] = new float* [8];
            for (short j = 0; j < 8; j++) ArrS[i][j] = new float[2] {};
        }

        short count_s = 0;
        GetSquare(protocol, ArrP, ArrS, N, count_s);

        float** ArrA = new float* [count_s * count_s];
        for (short i = 0; i < count_s * count_s; i++) ArrA[i] = new float[3];

        short count_a = 0;
        CalcArea(protocol, ArrS, ArrA, count_s, count_a);
        PrintRes(ArrA, ArrS, count_s, count_a, N, result);

        for (short i = 0; i < count_s * count_s; i++) delete[] ArrA[i]; delete[] ArrA;
        for (short i = 0; i < 4 * N; i++) {
            for (short j = 0; j < 8; j++) delete[] ArrS[i][j];
            delete[] ArrS[i];
        } delete[] ArrS;
        for (short i = 0; i < N0; i++) delete[] ArrP[i]; delete[] ArrP;
    }
}
```

Вывод программы

Контрольный пример: входной файл (рис. 5), файл с результатом (рис. 6), протокол (рис. 7).

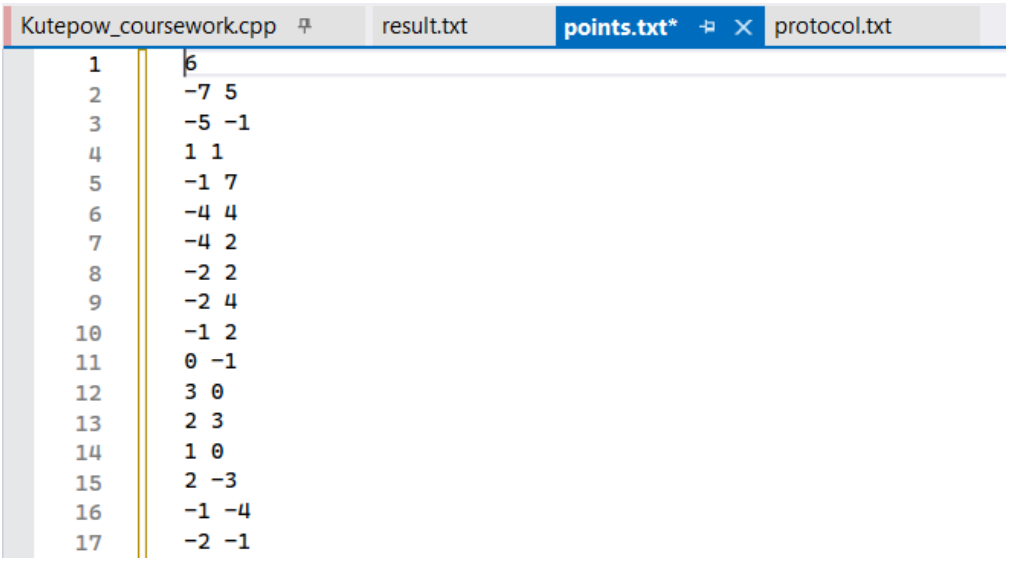


Рисунок 5

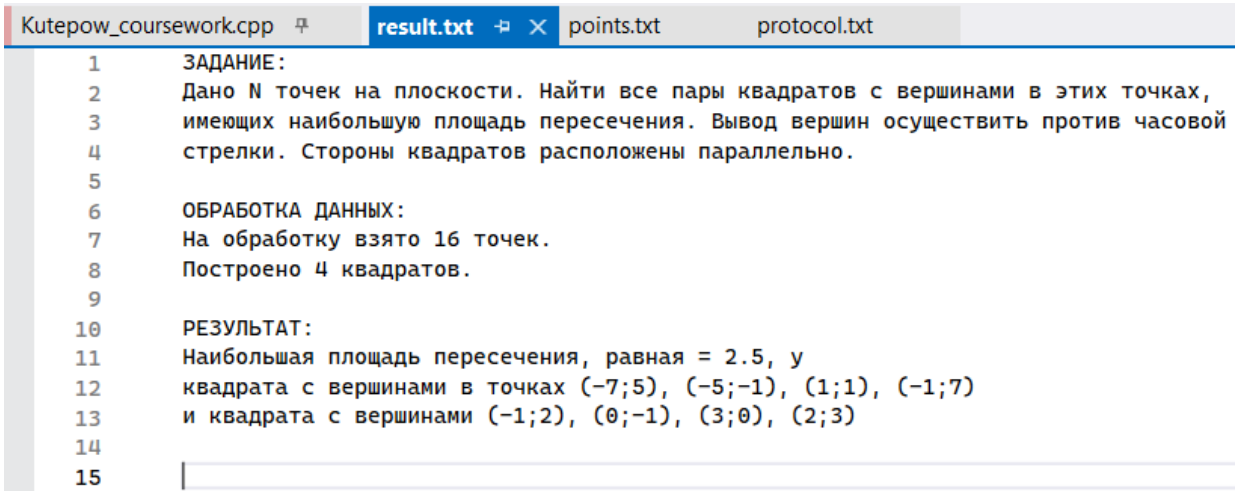


Рисунок 6

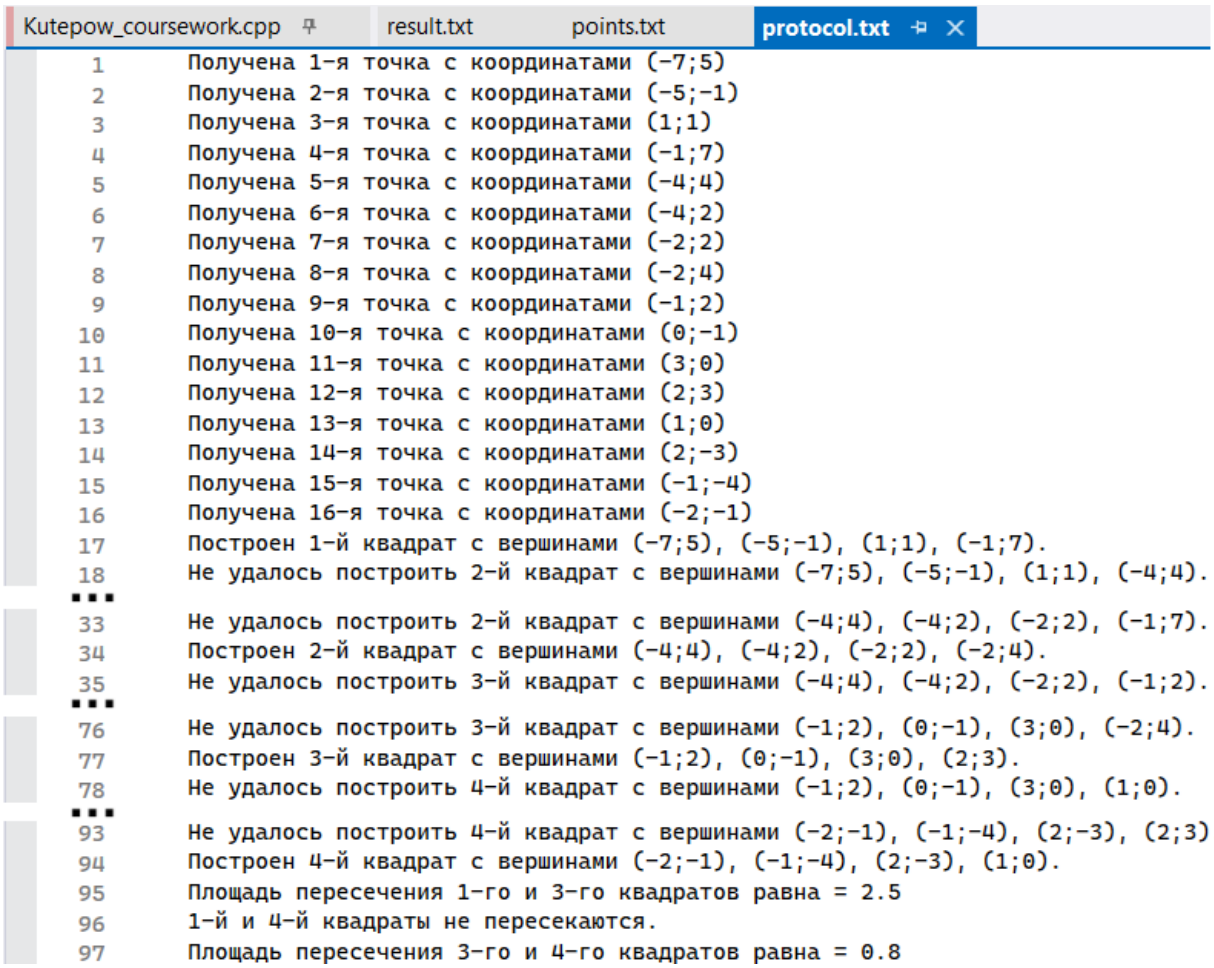


Рисунок 7

Выводы

В ходе данной курсовой работы были рассмотрены следующие понятия: обработка и хранение числовых данных, файловые переменные, режимы открытия и закрытия файла, проверка на то, открыт ли файл, считывание и

запись информации в файл, динамическая память, указатели, арифметика указателей, удаление данных, одномерные, двумерные и трёхмерные массивы, функции, параметры функций, возвращаемое значение функций, ссылки, типы данных, условные конструкции, циклы и функция main. Также на практике была реализована программа, решающая геометрическую задачу на пересечение фигур на координатной плоскости.